

Relational Division in Double-Categorical OLOGs

Michael Lambert

Spring 2026

Abstract

Working draft of a paper on quantification, modality, data queries and comprehension, all in the context of double categories. A follow-up to [LP25].

1 Introduction

2 Relational Division, Quantification, and Modality

Relational division is a query introduced in [Cod72, §2] that behaves essentially as a universal quantifier over given specified values in the column of a table. First we illustrate this operation with a very simple generic example. We may as well work with binary relations $R: C \leftrightarrow A$ which are of course equivalently monic arrows $\langle d, c \rangle: R \rightarrow C \times A$. Recall the notation from the reference

$$r[A] = c(r) \text{ where } r \in R$$

and likewise for $r[C] = d(r)$. This is a bit notationally-heavy, but is consistent with the reference and allows for generalizations beyond the following example. Likewise, for any binary relation T , let $g_T(x) = \{y \mid xTy\}$, that is, the set of all y in the target of T related to x under T . Call this the T -orbit of x . Now, for the example, consider the two relational tables R and S with columns (C, A) and (B, D) , respectively:

R		S	
C	A	B	D
1	a	a	2
1	b	b	1
1	c	b	2
2	a	b	2
2	b	a	3
3	b		

Thus, for example, $2Ra$ and $2Rb$. Likewise, $aS2$, et cetera. Now, in the notation of the reference, a division query is

$$R[A \div B]S = \{r[C] \mid r \in R \wedge S[B] \subset g_R(r[C])\} = \{1, 2\}$$

We unpack this carefully to illustrate the operation. Note that the columns A and B must take the same types of values although the values do not need to have the same exact extension. Now, $r[C]$ is the set of values of the R -column, C . So, $r[C] = \{1, 2, 3\}$. Likewise, $S[B] = \{a, b\}$. So, there are

three sets of the form $g_R(r[C])$, only two of which contain $S[B]$, as in:

$$\begin{aligned} S[B] &= \{a, b\} \not\subseteq \{b\} = g_R(3) \\ S[B] &= \{a, b\} \subset \{a, b, c\} = g_R(1) \\ S[B] &= \{a, b\} = \{a, b\} = g_R(2). \end{aligned}$$

Hence again, the division query returns $\{1, 2\}$, exactly *the set of all C -values whose orbit contains all the values of the B -column of S* . More prosaically, a C -value is selected by the query iff it is related to everything on the list of B -values. The universal quantification is thus over the set of B -values. The query allows the comparison of separate tables; note that D was not needed to do the query. In the containment relations above, the universally quantified set does not need to completely describe the orbit of any given element satisfying the division query; that is, 1 is related to c , but this is not a B -value. Likewise 3 is related to something on the list, but not everything.

A more concrete example illustrating the same query is the following. Consider the two relational tables below. On the left, rows pair an apothecary with an ingredient sold there; on the right, rows pair an ingredient with an effect produced by the combination of that ingredient with another in a potion.

Apothecary	
Name	Ingredient
Arcadia's Cauldron	giant's toe
Arcadia's Cauldron	wheat
Arcadia's Cauldron	void salts
Elgrim's Elixers	giant's toe
Elgrim's Elixers	wheat
The White Phial	wheat

Potion	
Ingredient	Effect
giant's toe	fortify health
wheat	fortify health
wheat	lingering damage magicka
giant's toe	damage stamina regen

Note that the second table does not list the other ingredients required to create the potion producing the effect. Our expertise is that any two such ingredients with the same effect will create the desired potion when combined. Now, suppose we want to create a fortify health potion but are too lazy to fight a giant to get his toe. We'd also really like to one-stop-shop (maybe the merchant will give us a deal if we buy in bulk). The orbits are then the inventories:

$$\begin{aligned} g_{\text{Apoth}}(\text{Arcadia's}) &= \{\text{giant's toe}, \text{wheat}, \text{void salts}\} \\ g_{\text{Apoth}}(\text{Elgrim's}) &= \{\text{giant's toe}, \text{wheat}\} \\ g_{\text{Apoth}}(\text{White Phial}) &= \{\text{wheat}\} \end{aligned}$$

The the set of values quantified over is our shopping list: $\{\text{giant's toe}, \text{wheat}\}$. The query is then

$$\text{Apothecary}[\text{Ingredient} \div \text{Ingredient}] \text{Potion} = \{\text{Name} \mid \{\text{giant's toe}, \text{wheat}\} \subset \text{Inventory}(\text{Name})\}$$

which is just to say that we are looking for the proprietors who have all the ingredients on our shopping list. The result is $\{\text{Arcadia's}, \text{Elgrims}\}$. It does not matter that Arcadia's also has void salts; but it does matter that The White Phial only has wheat but no giant's toe.

There is something awkward about these examples inasmuch as we would probably more likely

have tables better reflecting our alchemical expertise such as

Apothecary	
Name	Ingredient
Arcadia's Cauldron	giant's toe
Arcadia's Cauldron	wheat
Arcadia's Cauldron	void salts
Elgrim's Elixers	giant's toe
Elgrim's Elixers	wheat
The White Phial	wheat

Potion	
Ingredient	Effect
giant's toe	fortify health
wheat	fortify health
wheat	lingering damage magicka
giant's toe	damage stamina regen
swamp fungal pod	restore health
void essence	restore health

However, division with quantification over the whole the Ingredient column in the Potion table now produces the empty set since no merchant has all those items on hand. And [Cod72, §2.4] recognizes this in the example queries. Namely, example query #8 to find suppliers supplying at least parts 12, 13 and 15 restricts the part # values to only 12, 13 and 15 without ranging over the whole column before executing the division query. This is exactly a *shopping list* kind of example. In other words, technically, the division query is not set up to do selection from a sub-list of a column on its own. A restriction to just the values of interest from the second table is employed implicitly before doing the division query. In the example immediately above, to execute our query, we would filter the rows with the value “fortify health” first and then do the division query with the ingredients column. The flexibility of the division query as written is that it can be applied to any two tables with columns having the same types of values; the expense is that it might not return anything and may require restrictions to produce results in intuitive cases.

3 OLOGs and Instances

What is going on here from a double-categorical OLOG-perspective [LP25] is that we have two basic relations

$$\boxed{\text{Name}} \xrightarrow{\text{Apothecary}} \boxed{\text{Ingredient}} \quad \boxed{\text{Ingredient}} \xrightarrow{\text{Potion}} \boxed{\text{Effect}} .$$

Suppose that these relations are instantiated by the two tables at the end of the previous section, thought of as living in $\mathbb{R}\text{el}$. Now, the OLOG generated will be a cartesian equipment. So, given an individual as a global element of a type, say, Arcadia's: $1 \rightarrow \boxed{\text{Name}}$, we can form the restriction

$$\begin{array}{ccc} 1 & \xrightarrow{\text{Arcadia's Inventory}} & \boxed{\text{Ingredient}} \\ \text{Arcadia's} \downarrow & \text{res} & \downarrow \\ \boxed{\text{Name}} & \xrightarrow{\text{Apothecary}} & \boxed{\text{Ingredient}} \end{array}$$

which in the instance is precisely the named merchant's inventory. This would play the role of $g_R(x)$ in the previous section. Likewise, our ingredient list for a fortify health potion is schematized by a restriction:

$$\begin{array}{ccc} \boxed{\text{Ingredient}} & \xrightarrow{\text{fortify health potion}} & 1 \\ \downarrow & \text{res} & \downarrow \text{fortify health} \\ \boxed{\text{Ingredient}} & \xrightarrow{\text{Potion}} & \boxed{\text{Effect}} \end{array}$$

These are both examples of “filter” queries discussed in [LP25]. Now, we are trying ultimately to compare the two restrictions, to see if the ingredients for the fortify health potion are in the various inventories. In the examples above the composites

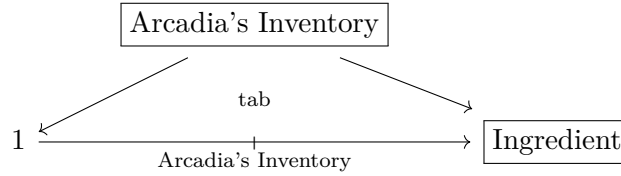
$$1 \xrightarrow{\text{Arcadia's Inventory}} \boxed{\text{Ingredient}} \xrightarrow{\text{fortify health potion}} 1$$

and

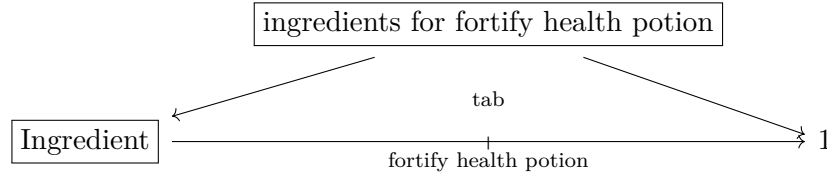
$$\boxed{\text{Name}} \xrightarrow{\text{Apothecary}} \boxed{\text{Ingredient}} \xrightarrow{\text{fortify health potion}} 1$$

are natural to study but neither returns what we want. In the first case, the $\mathbb{R}\text{el}$ -composite will return whichever ingredient Arcadia’s has on the list; in the second case, the $\mathbb{R}\text{el}$ -composite returns whichever merchants have an ingredient on the list. This is a result of the fact that existential quantification over the instance of the middle type is built into composition in $\mathbb{R}\text{el}$.

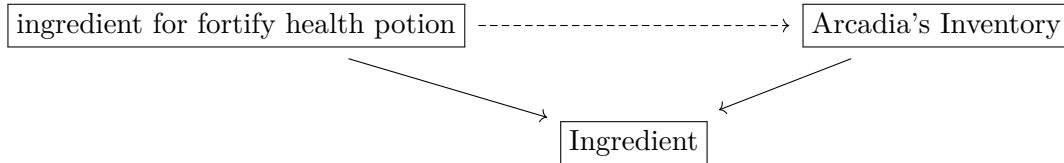
If we allow ourselves tabulators in the OLOG, much more progress can be made. This allows the creation of types with projections and universal cells, namely,



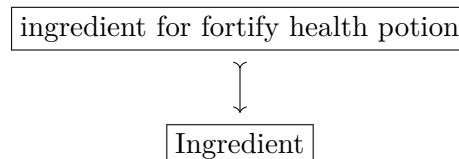
and then another



Now, the shopping query is thus to ask whether for each such inventory, the data reflects a putative relationship schematized as

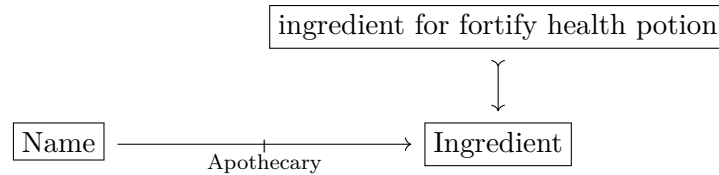


where, in other words, the inventory in question contains the ingredients on the list. Of course this is not quite right, because we’d like to look at all such inventories simultaneously and pick those merchants whose inventories contain all the ingredients. All the inventories will be contained in our instanting of the ingredient type. So, a better way to do this is just to isolate the type of ingredients and view our shopping list as a subtype of the type of ingredients:

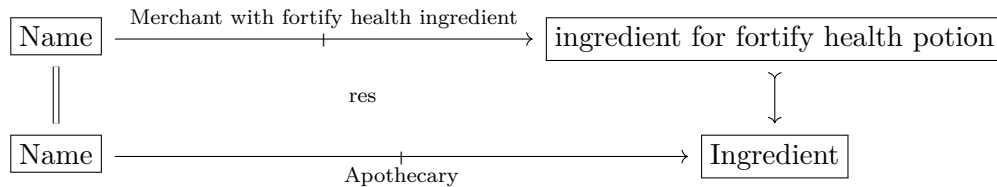


This doesn't necessarily need to be monic. But monic arrows are perfectly well axiomatizable in our framework. Perhaps we use the notation to indicate we are thinking of it as a subtype obtained by comprehension [Jac99, §9.3]. Nor does it matter how the subtype comes about at this stage; it could have been obtained as a tabulator as above, or simply required as a primitive in the setup of the OLOG.

What matters is comparing this subtype with the inventories of all the merchants simultaneously. These are implicit in the instantiation of the Apothecary relation, so we study the corner



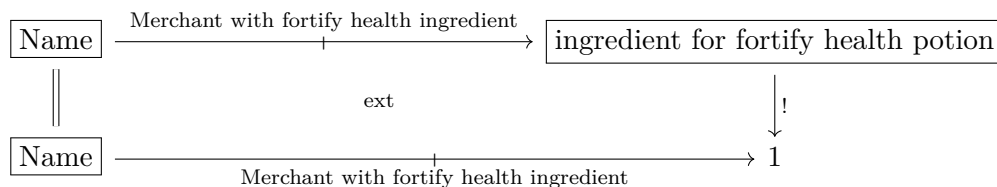
It is tempting just to restrict with an identity morphism on the left. This results in a cartesian cell



which filters the table for those rows having one of the two ingredients on our list:

Apothecary	
Name	Ingredient
Arcadia's Cauldron	giant's toe
Arcadia's Cauldron	wheat
Elgrim's Elixers	giant's toe
Elgrim's Elixers	wheat
The White Phial	wheat

Notice that this gets rid of the unnecessary entry with the value “void salts” but it does not establish our query. Another operation with which our OLOG comes equipped is extension. In $\mathbb{R}el$ this acts as an image, and formalizes a select query [LP25]. In the present instance we might try an extension



which in the data instance is effectively just a list of the three merchants, since each has at least one ingredient. But if we went to the White Phial, we'd be out of luck. This result is owing to the fact that in $\mathbb{R}el$ the extension is an image computed via *existential quantification*. So, the composite query here of restriction along the subtype, followed by extension has performed an *existential query*, returning each merchant whose inventory contains an ingredient on our list.

Now, this is clearly not the result we want, but it is not a dead-end, for it suggests a path forward. Extensions are essentially left adjoints to restriction. Inasmuch as extension is thought

of as existential quantification and restriction is thought of as substitution, we have an analogy of adjunctions

$$\text{ext} \dashv \text{res} \quad \leftrightarrow \quad \exists \dashv (-)^*.$$

Now, of course in nice logical situations substitution also has a right adjoint, which is *universal quantification*. This is of course exactly the kind of operation we are seeking to formalize. What we want then is to complete the adjoint analogy

$$\text{ext} \dashv \text{res} \dashv ??? \quad \leftrightarrow \quad \exists \dashv (-)^* \dashv \forall$$

in the theater of OLOGs and of double categories generally. Something like this has been studied in the foundational paper [Shu08] under the guise of *coextension*. It was mentioned there, but not studied in-depth due to being not as well behaved as the other two operations. And indeed this is not what we envision in this paper. For coextensions are had by asking for the equipment $\mathbb{D}_1 \rightarrow \mathbb{D}_0 \times \mathbb{D}_0$ to be also a *cofibration* which is too strong for what we have in mind. Rather, we take the approach of *fibrational semantics* [Jac99] and ask merely that our equipments $\mathbb{D}_1 \rightarrow \mathbb{D}_0 \times \mathbb{D}_0$ have right adjoints to restriction functors and that these satisfy a Beck-Chevalley condition. This is exactly to require *fibered products*. We shall see in more detail how this recovers exactly the queries of interest below in Section 6. For this we first review the operations of restriction and extension and then discuss the easier case of the formalism of existential queries in more depth.

4 Operations in Double Categories

Example 4.1. In $\mathbb{Mat}(\mathbf{V})$, restrictions and extensions are computed as

$$\begin{array}{ccc} Z & \xrightarrow{f_! M g^*} & W \\ f \downarrow & \text{res} & \downarrow g \\ X & \xrightarrow{M} & Y \end{array} \quad f_! M g^*(z, w) = M(fz, gw)$$

and

$$\begin{array}{ccc} Z & \xrightarrow{N} & W \\ f \downarrow & \text{ext} & \downarrow g \\ X & \xrightarrow{f^* N g_!} & Y \end{array} \quad f^* N g_!(x, y) = \sum_{\substack{fz=x \\ gw=y}} A(z, w)$$

□

5 Existential Queries

A relation $R: X \rightarrowtail Y$ might be thought of as a predicate, as a generalized many-valued non-functional relationship, or when $X = Y$ as a non-deterministic system. That x and y are R -related is written xRy or perhaps $x \leq_R y$ or even $x \sim_R y$. That in any case y is on the left suggests, due to non-symmetry, that y is somehow subsequent to, or a consequence of x . This comports with the system view of R that y is a later state accessible from x . Perhaps x is prior to y in a temporal sequence. On the causal view, somehow y is a consequence of x .

A proposition is a subset $S \subset Y$ or perhaps a monic arrow $S \rightarrowtail Y$. Such propositions are in 1-1 correspondence with functions $X \rightarrow \mathbf{2}$. That $x \in X$ satisfies the proposition $S \subset X$ is written $S(x)$

and means $x \in S$ too. Any given such x may or may not satisfy S . That some elements of X does so is usually written ' $\exists x S(x)$ ' and the operator ' $\exists$ ' binds the dummy variable. The set of all $x \in X$ for which some subsequent state satisfies $S \subset X$ is written

$$\Diamond S = \{x \mid S(y) \text{ for some } x \leq_R y\}.$$

This is exactly the modal possibility operator familiar from the topological semantics given by the Alexandrov Topology. Likewise there is the set of all elements of X , all of whose subsequent states satisfy $S \subset X$, written

$$\Box S = \{x \mid S(x) \text{ for all } x \leq_R y\}.$$

This is modal necessity in the same topological semantics. Thus, in the former case, an element of $\Diamond S$ is an element of X for which S is possible, in the sense that it is true of some later accessible state. In the latter case, an element of $\Box S$ is an element of X for which S is necessarily true, in the sense that any subsequent accessible state satisfies S .

The possibility set $\Diamond S$ above is constructed via the application of adjoint functors. That is, consider the proposition $s: S \rightarrowtail X$ as a monic arrow and form the pullback on the left followed by the image

$$\begin{array}{ccc} R \odot s^* & \xrightarrow{\quad} & R \\ \downarrow & \lrcorner & \downarrow \\ X \times S & \xrightarrow{1 \times s} & X \times Y \end{array} \rightsquigarrow \begin{array}{ccc} R \odot s^* & \xrightarrow{\quad} & X \times S \xrightarrow{1 \times !} X \times 1 \\ & \searrow & \nearrow \\ & \Diamond S & \end{array}$$

That is, the corner of the pullback on the right is exactly the set of pairs (x, a) where $x \sim_R a$ under R . The image then binds $x \in X$ via existential quantification. This is formalized in the double category $\mathbb{R}el$ of sets, functions and relations, $\Diamond S$ by first taking a restriction and then an extension:

$$\begin{array}{ccc} X & \xrightarrow{R \odot u^*} & S \\ 1 \downarrow & \text{res} & \downarrow s \\ X & \xrightarrow{R} & X \end{array} \rightsquigarrow \begin{array}{ccc} X & \xrightarrow{R \odot u^*} & S \\ 1 \downarrow & \text{ext} & \downarrow ! \\ X & \xrightarrow{\quad} & 1 \\ & \Diamond S & \end{array}$$

since restrictions in this double category are pullbacks and extensions are images.

Note that the same effect could be achieved simply by composing. That is, in any double category $\mathbb{D}$, there is a comparison cell of the form

$$\begin{array}{ccccc} X & \xrightarrow{R \odot s^*} & S & \xrightarrow{\text{id}} & S \\ 1 \downarrow & \text{res} & \downarrow s & \text{ext} & \downarrow ! \\ X & \xrightarrow{R} & X & \xrightarrow{S} & 1 \end{array}$$

where $S: X \rightarrowtail 1$ is defined via the extension cell and a slight abuse of notation. Thus, there is a globular cell $\Diamond S \Rightarrow R \odot S$ since $\Diamond S$ is constructed as an extension:

$$\begin{array}{ccc} X & \xrightarrow{R \odot s^*} & S \\ \downarrow & \cong & \downarrow \\ X & \xrightarrow{R \odot s^*} & S \\ 1 \downarrow & \text{res} & \downarrow s \\ X & \xrightarrow{R} & X \end{array} \begin{array}{ccc} & \xrightarrow{\text{id}} & S \\ & \text{ext} & \downarrow ! \\ & \Diamond S & \\ & \exists ! & \\ & \xrightarrow{R \odot S} & 1 \end{array} = \begin{array}{ccc} X & \xrightarrow{R \odot u^*} & S \\ \parallel & \text{ext} & \downarrow ! \\ X & \xrightarrow{\quad} & 1 \\ \parallel & \Diamond S & \\ \parallel & \exists ! & \\ X & \xrightarrow{R \odot S} & 1 \end{array}$$

Now, this is an invertible cell when $\mathbb{D}$ is either $\mathbb{R}el$ or $\mathbb{S}pan$ or $\mathbb{M}at(\mathbf{V})$ for suitably structured monoidal $\mathbf{V}$. The case of $\mathbb{R}el$ is trivial, so we treat the other two briefly.

Example 5.1. In $\mathbb{S}pan$, work with $\langle d, c \rangle: R \rightarrow X \times Y$ and then on the one hand

$$\diamond S = \{((x, a), r) \mid x \in X, a \in S, r \in R, d(r) = x, c(r) = a\}$$

whereas $R \odot \hat{s}$ is the corner of the pullback:

$$R \times_X S = \{(r, a) \mid r \in R, a \in S, c(r) = a\}.$$

These are evidently isomorphic, as the latter just forgets the indices $x \in X$. But these are recoverable as $d(r) = x$. $\square$

Example 5.2. Let $\mathbf{V}$ denote any monoidal category with coproducts preserved by the tensor. We then have the double category $\mathbb{M}at(\mathbf{V})$ of $\mathbf{V}$ -matrices. To form the possibility operator, we have first the restriction

$$\begin{array}{ccc} X & \xrightarrow{M \odot s^*} & S \\ \parallel & \text{res} & \downarrow s \\ X & \xrightarrow{M} & X \end{array} \quad (M \odot s^*)(x, a) = M(x, a) \quad \text{for } x \in X, a \in S$$

and then via an extension we have the possibility operator:

$$\begin{array}{ccc} X & \xrightarrow{M \odot s^*} & S \\ \parallel & \text{ext} & \downarrow ! \\ X & \xrightarrow{\diamond S} & ! \end{array} \quad \diamond S(x) = \sum_{a \in S} M(x, a) \quad \text{for } x \in X.$$

Now, to compare this to the composite $M \odot S$ note first that S viewed as a proarrow $S: X \rightarrow 1$ has the entries described by

$$\begin{array}{ccc} S & \xrightarrow{\text{id}_S} & S \\ s \downarrow & \text{ext} & \downarrow ! \\ X & \xrightarrow{S} & 1 \end{array} \quad S(x) = \begin{cases} I & x \in S \\ 0 & \text{o/w} \end{cases} \quad \text{for } x \in X.$$

This follows from the definition of the unit proarrows in $\mathbb{M}at(\mathbf{V})$. Now, from the definition of the loose composite of two matrices, we have

$$\begin{aligned} (M \odot S)(x) &= \sum_{y \in X} M(x, y) \otimes S(y) \\ &= \sum_{y \in X} M(x, y) \otimes \begin{cases} I & y \in U \\ 0 & \text{o/w} \end{cases} \\ &= \sum_{y \in S} M(x, y) \end{aligned}$$

for any $x \in X$, showing that the two constructions are the same in this case too. $\square$

However, in $\mathbb{P}rof$, the cell under consideration is not invertible. This is essentially a failure of left Kan extension to commute with composition of profunctors.

Example 5.3. □

Since in general the constructions are not the same, we must make a choice as to which is basic. In the case of $\mathbb{S}pan$, although the appropriate element $x \in X$ is implicit in the latter as $d(r) = x$, our preference is to keep the data as a triple $((x, a), r) = (x, a, r)$ as in the first way of computing the possibility operator. This way all the information is recorded in the apex of the span and the two legs can be forgotten. The preferred definition is thus formalized in the following way.

Definition 5.4 (Possibility). Let $s : S \rightarrowtail X$ denote any monic arrow in the equipment $\mathbb{D}$. The proposition **possibly** S is then formed as an restriction followed by an extension:

$$\begin{array}{ccc} X & \xrightarrow{R \odot u^*} & S \\ 1 \downarrow & \text{res} & \downarrow s \\ X & \xrightarrow{R} & X \end{array} \quad \rightsquigarrow \quad \begin{array}{ccc} X & \xrightarrow{R \odot u^*} & S \\ 1 \downarrow & \text{ext} & \downarrow ! \\ X & \xrightarrow{\diamond S} & 1 \end{array}$$

as in the pattern suggested above: □

This operator justly bears the ‘ $\diamond$ ’ notation on account of the following observation, namely, that what is the case is possibly the case, as in the usual monadic interpretation of possibility. The proof is most naturally interpreted in $\mathbb{R}el$, but is completely formal. When $\mathbb{R}el$ is meant, the relationship is the familiar $S \leq \diamond S$.

Proposition 5.5. *If R is reflexive, then there is a globular cell $S \Rightarrow \diamond S$.*

Proof. Reflexivity of R means that there is a canonical cell

$$\begin{array}{ccc} X & \xrightarrow{\text{id}_X} & X \\ \parallel & \delta_X & \parallel \\ X & \xrightarrow{R} & X \end{array}$$

expressing the fact that the diagonal factors through R . Consequently, since $R \odot u^*$ arises as a restriction, there is a unique cell making an equality

$$\begin{array}{ccc} S & \xrightarrow{\text{id}_S} & S \\ s \downarrow & \text{id}_s & \downarrow s \\ X & \xrightarrow{\text{id}_X} & X \\ \parallel & \delta_X & \parallel \\ X & \xrightarrow{R} & X \end{array} = \begin{array}{ccc} S & \xrightarrow{\text{id}_S} & S \\ s \downarrow & \exists ! & \parallel \\ X & \xrightarrow{R \odot s^*} & S \\ \parallel & \text{res} & \downarrow s \\ X & \xrightarrow{R} & X \end{array}$$

And so, putting this cell as (I) in the diagram below, we get the desired comparison induced by the

universal property of the extension cell:

$$\begin{array}{ccc}
\begin{array}{ccc}
S & \xrightarrow{\text{id}_S} & S \\
s \downarrow & \text{(I)} & \parallel \\
X & \xrightarrow{R \odot s^*} & S \\
\parallel & \text{ext} & \downarrow ! \\
X & \xrightarrow{\diamond_S} & 1
\end{array} & = & \begin{array}{ccc}
S & \xrightarrow{\text{id}_S} & S \\
s \downarrow & \text{ext} & \downarrow ! \\
X & \xrightarrow{S} & 1 \\
\parallel & \exists ! & \parallel \\
X & \xrightarrow{\diamond_S} & 1
\end{array}
\end{array}$$

Notice the abuse of notation, namely, that we think of S as both the object providing the domain of s but also as a proarrow $S: X \rightarrowtail 1$ obtained by extension. $\square$

6 Universal Queries

7 Tabulators and Modality

8 Full Closed Comprehension

9 Prospectus

References

- [Cod72] E.F. Codd. “Relational Completeness of Data Base Sublanguages”. *Research Report / RJ / IBM / San Jose, California* RJ987 (1972).
- [Jac99] Bart Jacobs. *Categorical logic and type theory*. Elsevier, 1999. DOI: 10.1016/S0049-237X(98)80028-1.
- [LP25] Michael Lambert and Evan Patterson. “Representing knowledge and querying data using double-functorial semantics”. *Proceedings of the Seventh International Conference on Applied Category Theory (ACT 2024)*. 2025, pp. 174–189. DOI: 10.4204/EPTCS.429.9. arXiv: 2403.19884.
- [Shu08] Michael Shulman. “Framed bicategories and monoidal fibrations”. *Theory and Applications of Categories* 20.18 (2008), pp. 650–738. arXiv: 0706.1286.